

Visual attention and working memory

A textbook companion

KDA, spatial associative memory,
recurrent readout, and all thirteen training tasks

Step-by-step derivations. Worked examples.
Architecture diagrams. Trial walkthroughs.
Exercises with answers.

24 SEPTEMBER 2026

The joint KDA architecture is implemented. The new final ConvGRU is described as a planned design under construction. This edition contains no new model experiments.

Contents

How to use this reading	3
1. The computation a trial requires	4
1.1 From pixels to an answer	4
1.2 Three kinds of persistence	4
1.3 What the model receives	4
1.4 A blank frame does not immediately remove the sample	5
2. KDA: learning how to revise an associative memory	6
2.1 The lineage and the local implementation	6
2.2 A matrix represents associations	6
2.3 Why plain addition is insufficient	6
2.4 Deriving the error-correcting write	7
2.5 Retention comes before the correction	7
2.6 A complete numerical step	8
2.7 Interference follows from key overlap	8
2.8 Retention is not a single fixed time constant	9
2.9 What the stability argument actually proves	9
2.10 The temporal attention hidden inside the recurrence	9
3. Spatial KDA: a field of small associative memories	10
3.1 What makes it spatial	10
3.2 How pixels become keys and gates	10
3.3 Local state does not imply an isolated pixel	10
3.4 State, emission and decision are different objects	10
3.5 Storage and computation costs	11
4. The implemented joint architecture	12
4.1 The visual hierarchy, one frame at a time	12
4.2 The final compression	12
4.3 The vector GRU's gates	12
4.4 Why more than one memory route matters	13
4.5 Reading the parameter count correctly	13
5. ConvGRU and the planned spatial readout	14
5.1 What is actually being changed	14
5.2 The ConvGRU equations	15
5.3 What recurrence can do with space	16
5.4 KDA and ConvGRU store different things	16
5.5 Three architectures that must not be confused	16
5.6 What the new experiment could teach us	17
6. The seven two-frame sensory tasks	18
6.1 What this group isolates	18
6.2 Motion direction: correspondence across two images	18
6.3 Signed orientation: compare axes, not pixel phases	18
6.4 Contrast: identify the stronger modulation	19
6.5 Spatial frequency: compare stripe density	19
6.6 Chromatic increment: a specified colour direction	19
6.7 Contour grouping: the arrangement is the signal	20

6.8 Natural spectral detail: a photograph task without identity memory	21
7. Six tasks with spatial instructions or longer sequences	22
7.1 Ring-cued orientation: location plus signed change	22
7.2 Signed cued orientation: apply a rule to the chosen location	23
7.3 Cued motion duration: count evidence across transitions	23
7.4 Krauzlis-style motion change: reject a real foil event	24
7.5 Spatial binding: remember which feature belonged where	24
7.6 Scene recognition: exact membership after a list	25
7.7 The complete condition map	25
8. How the tasks train a shared recurrent system	26
8.1 One final loss, many earlier computations	26
8.2 Effective batch size and microbatches	26
8.3 Equal tasks do not mean equal conditions or frames	26
8.4 Acquisition, validation and testing	27
8.5 Accuracy, balanced accuracy and AUC answer different questions	28
8.6 What should be in a learning report	28
9. From successful behavior to a mechanism	29
9.1 Availability and use are different questions	29
9.2 Three senses of attention	29
9.3 Perturbing an emission versus perturbing memory	29
9.4 What a controlled causal comparison needs	30
10. Exercises with worked answers	31
10.1 Work the associative update	31
10.2 Find the interference	31
10.3 Read a gate correctly	31
10.4 Identify the changed architectural hypothesis	31
10.5 Interpret a signed cue	31
10.6 Separate binding from global change	31
10.7 Audit an apparently impressive score	32
10.8 Count evidence, not repeated frames	32
10.9 Distinguish retained information from successful behavior	32
Reference sheet and source map	33
Notation at a glance	33
Implementation and design sources	33
Primary literature and documentation	34
What this edition establishes	34

How to use this reading

This is a teaching companion to *Spatial KDA memory: architecture and causal tests*, the ten-page note dated 23 September 2026. That note is organized around reporting and experimental recommendations. This book is organized around understanding: what computation each component performs, why it is there, what a complete trial asks the model to do, and what we can infer from its answer.

You are assumed to know neural-network training, convolution, matrix multiplication and probability. We will not rehearse those basics. We will unpack the less familiar machinery: associative matrix memory, error-correcting writes, channel-wise retention, spatial recurrence, recurrent readout, and the exact semantics of the task suite. Equations are followed by their operational meaning and, where useful, a small worked example.

Architecture status matters. The existing joint learner uses three spatial KDA modules followed by a vector GRU. The new branch being built by another agent retains those KDA modules and replaces the final vector-recurrent route with a spatial ConvGRU. This book describes that branch as a **planned design**, according to its recorded brief and your clarification. It makes no claim that the new branch is implemented, training, or better. Historical ConvGRU accumulator experiments are a third, different architecture.

The source PDF's behavioral numbers are a dated snapshot, not a live progress report. We preserve its scientific questions but do not turn its interim scores into current results. No training, model evaluation, checkpoint modification or intervention was performed to write this reading. Illustrative numbers below are calculations or constructed examples, explicitly distinguished from measured model behavior.

Read Chapters 1-5 consecutively for the architecture. Chapters 6-7 teach all thirteen tasks. Chapter 8 connects the tasks to learning and evaluation. Chapter 9 returns to the causal questions in the original note. Chapter 10 provides exercises and worked answers. The source map at the end identifies the implementation or design behind each account.

1. The computation a trial requires

1.1 From pixels to an answer

Consider a delayed orientation trial. A glyph indicates a location and a requested rotation sign. Four oriented patches appear. They disappear for several frames, then reappear. The required answer is whether the selected patch rotated in the requested sign.

A useful decomposition is **encode, select, retain, compare, report**. Encoding must represent orientation well enough to distinguish the relevant rotation. Selection must associate the instruction with the correct spatial patch. Retention must preserve useful sample information while the patch is absent. Comparison must relate that information to the probe. Reporting must map the comparison and instruction to the appropriate class.

Those are functional requirements, not five named modules in our code. A recurrent network can distribute one function across several states or use one state for several functions. For example, the final GRU might retain the cue, while KDA retains local appearance. Alternatively, the GRU might retain a partly computed answer. The architecture permits these possibilities; behavioral accuracy alone does not tell us which one was learned.

This decomposition prevents an easy mistake: calling every failure after a delay a memory failure. If the network never represented orientation accurately, perfect retention would preserve an inadequate representation. If it retained the right information but used the wrong location, the failure would look similar at the output. We need tasks and diagnostics that separate these explanations.

1.2 Three kinds of persistence

Learned parameters persist between trials. They include convolution kernels, the networks that generate KDA keys and gates, recurrent weights and task-head weights. During training, the optimizer changes them across batches. At inference, they define the rules of computation.

Recurrent activation state persists within a trial. KDA's matrices and the GRU hidden state change as frames arrive. They are reset between trials. They are not optimizer parameters. A matrix that changes at every frame is therefore not evidence that the model is changing its trained weights at inference.

The raw-frame stack makes the previous two images directly available with the current image. This is a fixed buffer, not a learned retention mechanism. Its influence must be separated from persistence in KDA or GRU state. [C1-C3]

A familiar associative-memory interpretation calls a rapidly updated matrix a *fast weight*. That term is useful only if we remember the distinction: here it is a trial-local activation computed by a learned rule, not an additional optimizer-trained parameter tensor.

1.3 What the model receives

The suite produces an image tensor of shape

$$X \in [0, 1]^{B \times T \times 3 \times 100 \times 100}.$$

Here B counts independent trials and T counts presented frames. A batch has one task and one condition, so all trials have the same native length. The model centers pixels by subtracting 0.5 and constructs

$$x_t = \text{concat}(X_{t-2} - 0.5, X_{t-1} - 0.5, X_t - 0.5).$$

Missing earlier frames are zero in centered coordinates. The encoder therefore receives nine channels at each step. This operation is causal: it never includes a future frame. On a two-frame task, there are still only two actual observations. Padding is not an extra stimulus. [C2-C4]

Only images enter the shared visual and recurrent computation. The experimenter selects the output head using the task ID. Numerical angles, target indices, phase labels, image identities and correct answers are analysis metadata, not input features. Visible instruction glyphs, however, are pixels and can be learned from. The model is not required to infer the externally selected task ID.

1.4 A blank frame does not immediately remove the sample

Suppose s is the final sample frame and b_1, b_2, b_3 are subsequent blanks. The first blank update still sees two earlier images; the second sees $[s, b_1, b_2]$; only the third sees $[b_1, b_2, b_3]$. Thus the first two nominal blanks still offer direct sample access through the stack.

For the ring task, the final probe step sees both sample frames and the probe together. That task can test spatially selective comparison without establishing learned retention through a long absence. At longer delays, the sample leaves the stack and useful information must reach the answer through other routes.

Blank processing is also not an identity operation. Convolution biases, instruction markers, normalization, KDA writes and recurrent dynamics can remain active. A model must learn appropriate behavior on blanks; it does not automatically freeze when the stimulus disappears.

Check your understanding. If a model is perfect at zero delay but fails after four blank frames, what has been established? There is a dependence on the longer trial or stimulus absence. We have not yet distinguished failed retention from changed state dynamics, cue loss, or an unfamiliar comparison context.

2. KDA: learning how to revise an associative memory

2.1 The lineage and the local implementation

KDA means **Kimi Delta Attention**. The Kimi Linear paper develops a gated delta-rule memory with fine-grained decay, within a larger language-model architecture. Our visual model adopts a small spatial version of the associative recurrence. It does not reproduce the full Kimi Linear architecture, its hybrid attention layers, or its optimized execution system. The equations below follow our implementation; the derivations and numerical examples explain that code. [C1; P1]

2.2 A matrix represents associations

At one site and one head, let the state be $S \in \mathbb{R}^{d_k \times d_v}$. A key $k \in \mathbb{R}^{d_k}$ addresses an association, while a value $v \in \mathbb{R}^{d_v}$ is the content associated with that key. Given a query $q \in \mathbb{R}^{d_k}$, the read is

$$o = S^\top q \in \mathbb{R}^{d_v}.$$

Why does this act like memory? Imagine first writing one outer product $S = kv^\top$. Then

$$S^\top q = v(k^\top q).$$

The query returns the value multiplied by its alignment with the key. If the normalized query equals the normalized key, it returns v . If the query is orthogonal to the key, this association contributes zero. Queries and keys are learned features, so their coordinates need not correspond to interpretable things such as “red,” “left,” or “orientation.”

For several additive writes,

$$S = \sum_i k_i v_i^\top, \quad S^\top q = \sum_i (k_i^\top q) v_i.$$

This is an associative retrieval rule. It resembles attention in that a query controls the contribution of stored content. These coefficients need not be positive or sum to one. They are not probabilities over locations, and storing this matrix does not preserve a separate record of every past image.

2.3 Why plain addition is insufficient

Suppose the same unit key repeatedly appears with the same value. The additive rule $S_t = S_{t-1} + k_t v_t^\top$ accumulates that value again each time. After ten identical writes from zero, querying that key returns ten times the original value. Sometimes accumulation is wanted; sometimes repeated observation should confirm a stable association without making it grow indefinitely.

The delta rule asks a more selective question: **What does memory currently predict at this key, and what correction is needed?** It writes the residual error rather than the whole observation. A repeated association that is already predicted well needs little additional correction.

This distinction is relevant to our tasks. Repeated sample frames need not be separate items. Repeated recognition probes should not automatically count as repeated study evidence. An error-based write creates the possibility of controlled revision, although the network must still learn representations and gates that use that possibility effectively.

2.4 Deriving the error-correcting write

For the moment, ignore decay. Define an instantaneous reconstruction objective

$$\ell(S) = \frac{1}{2} \|v - S^\top k\|_2^2.$$

Its gradient with respect to S is

$$\nabla_S \ell = k(S^\top k - v)^\top.$$

A step of size β in the negative-gradient direction gives

$$S_{\text{new}} = S + \beta k(v - S^\top k)^\top.$$

That is the delta-rule write. This derivation explains the form of the recurrence; our training loop does **not** separately minimize this local objective with another optimizer. The code directly computes the update. The global task loss trains the networks that produce k, v, q and the gates.

To see the correction precisely, set $a = S^\top k$. After the write,

$$S_{\text{new}}^\top k = a + \beta \|k\|_2^2 (v - a).$$

For a unit key this becomes $(1 - \beta)a + \beta v$. The retrieved prediction moves a fraction β toward the new value. With $\beta = 1$, it exactly fits that key's current value. With a very small β , it changes little. The implementation uses sigmoid gates, so exact zero and one are limiting cases rather than ordinary finite-logit outputs.

2.5 Retention comes before the correction

Our full recurrence has four operations:

$$\begin{aligned} \bar{S}_t &= D_{\alpha_t} S_{t-1}, & \hat{v}_t &= \bar{S}_t^\top k_t, \\ e_t &= v_t - \hat{v}_t, & S_t &= \bar{S}_t + \beta_t k_t e_t^\top, \\ o_t &= S_t^\top q_t. \end{aligned}$$

D_{α_t} is diagonal. Each of its d_k entries scales one row of the old matrix. The memory first retains or attenuates old associations, then predicts from that decayed state, then corrects the prediction, then reads the updated state. Reading after writing means the current observation can immediately influence the emission.

The two gates have different jobs. **Retention** α controls survival of existing state along key coordinates. **Write strength** β controls the size of the current error correction. A small write gate does not preserve memory if retention is also small. A large retention gate does not prevent a strong write from overwriting an overlapping association.

Retention is row-wise, not one independently controlled decay per value element. In our model there are eight retention values per head, and each scales a row containing sixteen value coordinates. The learned key basis determines what those rows mean.

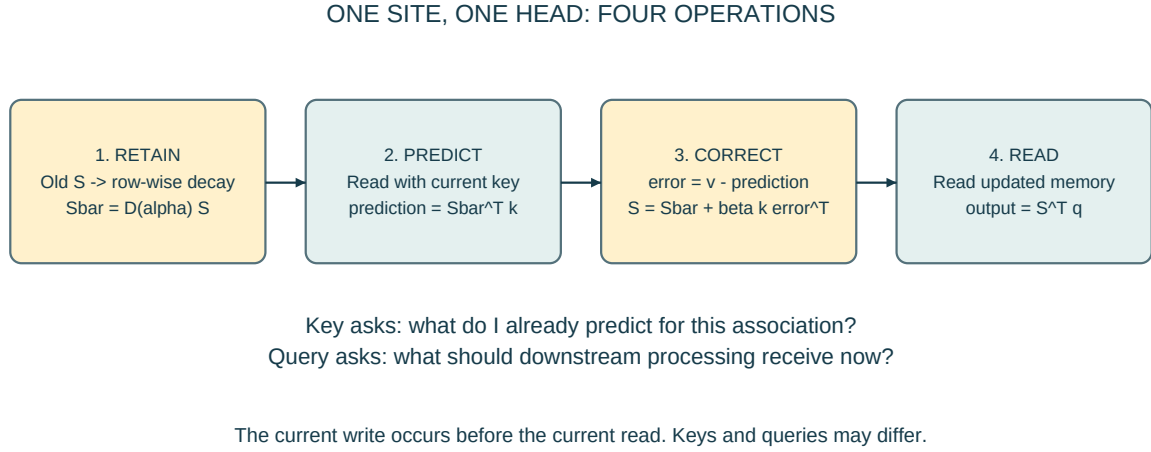


Figure 1: The associative update separates retention, prediction, correction and readout. Each operation has a different role.

2.6 A complete numerical step

Use a two-key, two-value toy memory, smaller than the deployed module:

$$S_{t-1} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad \alpha = (0.5, 1), \quad k = q = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v = \begin{bmatrix} 4 \\ 2 \end{bmatrix}, \quad \beta = 0.5.$$

Decay gives $\bar{S}_t = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$. The current key predicts $(1, 0)^\top$, so the error is $(3, 2)^\top$. The rank-one correction is

$$\beta k e^\top = \begin{bmatrix} 1.5 & 1 \\ 0 & 0 \end{bmatrix}.$$

Therefore

$$S_t = \begin{bmatrix} 2.5 & 1 \\ 0 & 3 \end{bmatrix}, \quad o_t = (2.5, 1)^\top.$$

The output is halfway between the decayed prediction $(1, 0)$ and the requested value $(4, 2)$. The second row is unaffected by this write because the key points entirely along the first coordinate. This is a constructed arithmetic example, not an extracted model state.

2.7 Interference follows from key overlap

Now ask what this write does to another query q' . Its immediate change relative to the decayed state is

$$\Delta o(q') = \beta (k^\top q') e.$$

Orthogonal queries are unaffected; aligned queries receive the full correction; negatively aligned queries receive a signed effect. There is no guarantee that one object occupies one orthogonal address. Learned keys can overlap, and a write for one stimulus can alter retrieval for another.

The matrix's rank is at most $\min(d_k, d_v)$. Our 8×16 matrix has rank at most eight. That does **not** mean it stores exactly eight objects. Object capacity depends on the representational code, interference, required precision, downstream readout and task distribution. "Eight key dimensions" is an architectural fact; "eight remembered items" would be an empirical claim requiring a capacity experiment.

2.8 Retention is not a single fixed time constant

If writes were absent and every row retained the same fixed fraction a , old state would decay as a^n after n steps. For illustration, $0.9^{24} \approx 0.080$ whereas $0.99^{24} \approx 0.786$. Small differences in retention can strongly affect long delays.

Our retention and write gates are input-dependent after training, and different rows can have different values. Blank frames can generate nonzero values and writes. Successive state changes therefore cannot generally be summarized by one exponential. Initial retention is 0.9 and initial write strength is 0.5, but these are initialization settings, not measured trained laws. [C1]

2.9 What the stability argument actually proves

Expanding the correction gives

$$S_t = A_t S_{t-1} + \beta_t k_t v_t^\top, \quad A_t = (I - \beta_t k_t k_t^\top) D_{\alpha_t}.$$

For fixed keys and gates, with $\|k_t\|_2 \leq 1$ and $0 \leq \beta_t \leq 1$, the rank-one factor has eigenvalue $1 - \beta_t \|k_t\|^2$ along the key and eigenvalue one in orthogonal directions. Its spectral norm is at most one. Hence

$$\|A_t\|_2 \leq \max_j \alpha_{t,j} \leq 1.$$

If two memories receive the *same fixed input-derived quantities*, their difference contracts or stays unchanged under this homogeneous transition. This is useful, but limited. It is not a proof that the complete network's gradients are bounded. Values are unconstrained; the input-driven term can add state; and the hierarchy contains other memory-dependent computations and a final recurrent route. Nor does nonexpansion guarantee accurate memory: a map that aggressively erases information is very stable.

2.10 The temporal attention hidden inside the recurrence

With $S_0 = 0$, unrolling the recurrence yields

$$S_t = \sum_{i=1}^t (A_t A_{t-1} \cdots A_{i+1}) \beta_i k_i v_i^\top.$$

An empty product is the identity. Querying gives

$$o_t = \sum_{i=1}^t w_{t,i} v_i, \quad w_{t,i} = \beta_i q_t^\top (A_t \cdots A_{i+1}) k_i.$$

Each past value contributes according to its write strength, its surviving address after intervening transitions, and the present query. These are **implicit temporal contributions** at one site/head. They need not sum to one, can be negative, and are not a softmax allocation over the image. Moreover, changing an actual past image changes downstream keys and gates too; this fixed-trajectory expansion is not by itself a causal attribution to pixels.

3. Spatial KDA: a field of small associative memories

3.1 What makes it spatial

A sequence KDA module maintains a matrix as tokens arrive. Our spatial adaptation maintains a separate matrix at every location of a feature map, updating those matrices as video frames arrive. Time is the recurrent axis; image positions remain explicit indices.

At scale s , location p and head h ,

$$S_{t,p,h}^{(s)} \in \mathbb{R}^{8 \times 16}, \quad h \in \{1, 2\}.$$

The full tensor is $B \times H_s \times W_s \times 2 \times 8 \times 16$. The two heads are separate learned associative systems at the same site. They are not left/right stimulus slots or experimenter-assigned feature types. Both follow the same algebra but have different learned projections. [C1-C2]

A site stores 256 scalars, but emits 32: two sixteen-dimensional queried values. Thus the downstream network does not receive the whole state matrix. It receives a query-dependent view of it. Information might remain in the state while the current query fails to expose it.

3.2 How pixels become keys and gates

The encoder block produces C_s channels. A learned 1×1 projection maps these to 32 channels at each location. A 3×3 convolution then generates 82 channels, packed as two groups of 41:

$$41 = 8 (q) + 8 (k) + 16 (v) + 8 (\alpha) + 1 (\beta).$$

Queries and keys are L2-normalized, using epsilon 10^{-6} to handle very small norms. The gates use sigmoids. Values have no such bound. After the update and read, the two head outputs are concatenated and transformed by a learned 1×1 convolution, including a bias. This yields the emitted field O_t . [C1]

This packing explains what “input-dependent gates” means concretely: changing a local input neighborhood changes convolution outputs, which can change retention and write strength. The same-scale previous matrix is not concatenated into that gate-producing convolution. It influences prediction and readout through the recurrence. At higher scales, current inputs already include lower-scale memory emissions, so higher-scale gating can be memory-conditioned indirectly.

3.3 Local state does not imply an isolated pixel

There is no direct term transporting $S_{t-1,p'}$ into $S_{t,p}$ from a neighboring site in the KDA update itself. Nevertheless, the quantities that control that update come from spatial convolutions. Each site represents a receptive field, not one raw pixel. Deeper scales combine broader neighborhoods, and GroupNorm uses spatial statistics within channel groups.

These facts matter for interpreting a map. A high gate value near a target is not automatically a neuron specifically coding that target. A perturbation applied to a few feature sites can affect downstream sites, normalization statistics and the final decision. “Spatially indexed” describes how the state is organized; it does not certify strict causal independence among regions.

3.4 State, emission and decision are different objects

The **state** is the matrix carried to the next frame. The **raw read** is $S_t^\top q_t$. The **emission** applies the learned output projection to the concatenated reads. The **decision representation** is computed after further convolution and recurrence.

These distinctions explain several otherwise puzzling observations. Zero state does not necessarily imply zero emission, because the output projection has a bias. A large state norm can produce a small read if the query is nearly orthogonal to its important directions. A visible emission change can be ignored by downstream weights. Conversely, a small targeted feature change can have a large decision effect near a classification boundary.

None of these quantities is a universal “attention strength.” Gate magnitude, state norm, query-key alignment, feature influence and behavioral selection describe different aspects of computation. A textbook account should make it possible to say which one is being plotted.

3.5 Storage and computation costs

The three KDA grids contain $25^2 + 13^2 + 7^2 = 843$ sites. Multiplying by 256 gives **215,808 state scalars per example**, or 863,232 bytes in fp32. This excludes frame buffers, the final recurrent state, intermediate activations and optimizer storage.

Each KDA module has $32 \times 82 \times 3 \times 3 + 82 = 23,698$ input-convolution parameters and $32 \times 32 + 32 = 1,056$ output-projection parameters: **24,754 learned parameters per scale**. The input projection from C_s to 32 is separate. These arithmetic counts follow the source shapes. [C1-C2]

At fixed image resolution, recurrent state size does not grow with sequence length. Full backpropagation through a longer trial still needs more training-time intermediates. Constant-size forward state therefore does not mean constant training memory or constant work for an entire sequence.

4. The implemented joint architecture

4.1 The visual hierarchy, one frame at a time

The existing joint learner uses the following route. “Visual” is the encoder output before concatenation; “combined” is what the next stage receives. Every learned component is trainable. [C2-C3]

Stage	Visual output	Memory emission	Combined output
Conv 1	$32 \times 50 \times 50$	None	$32 \times 50 \times 50$
Conv 2	$64 \times 25 \times 25$	$32 \times 25 \times 25$	$96 \times 25 \times 25$
Conv 3	$96 \times 13 \times 13$	$32 \times 13 \times 13$	$128 \times 13 \times 13$
Conv 4	$128 \times 7 \times 7$	$32 \times 7 \times 7$	$160 \times 7 \times 7$

All convolutions have stride two. The first kernel is 5×5 ; the others are 3×3 , with padding. Each block uses eight-group GroupNorm followed by ReLU. GroupNorm here is an encoder operation; it does not directly normalize every KDA state matrix.

The order is important. At a given frame, Conv 2 is followed by the first KDA update and read. That emission becomes part of the input to Conv 3, whose output drives the next KDA. Processing then repeats at Conv 4. Memory therefore participates inside the hierarchy, rather than being appended only after all visual computation is finished.

4.2 The final compression

Let $F_t \in \mathbb{R}^{160 \times 7 \times 7}$ be the deepest combined map. The old route is

$$f_t = \text{ReLU}(W_f \text{vec}(F_t) + b_f) \in \mathbb{R}^{256},$$

$$h_t = \text{GRU}(f_t, h_{t-1}) \in \mathbb{R}^{256}, \quad \ell = W_{\text{task}} h_T + b_{\text{task}}.$$

The linear projection compresses 7,840 map coordinates to 256 features **before** the final recurrent update. Flattening does not remove location labels in the way global average pooling would: different positions get different learned weights. But the final recurrent state has no explicit spatial grid, and each time step’s map must pass through that compressed representation.

This motivates a readout hypothesis. Perhaps the spatial memories contain useful information, but the final route does not preserve or compare it adequately. It remains a hypothesis. Compression can also help by extracting sufficient task statistics and removing irrelevant detail. A dimensional bottleneck alone is not evidence that the model fails because of that bottleneck.

4.3 The vector GRU’s gates

A GRU combines a previous state with a candidate. In the PyTorch convention used by the existing `nn.GRU`, the update gate z_t weights the **old** state:

$$r_t = \sigma(W_{ir} f_t + b_{ir} + W_{hr} h_{t-1} + b_{hr}),$$

$$z_t = \sigma(W_{iz} f_t + b_{iz} + W_{hz} h_{t-1} + b_{hz}),$$

$$n_t = \tanh(W_{in} f_t + b_{in} + r_t \odot (W_{hn} h_{t-1} + b_{hn})),$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.$$

The reset gate controls the recurrent contribution to the candidate; the update gate controls mixing with the old state. PyTorch applies the reset multiplication after the hidden affine transform in the candidate. This detail differs from placing the reset on the hidden state before the transform. [P3]

The proposed ConvGRU will use a **write** gate instead, where a larger value weights the candidate more. When comparing plots or equations, compare gate roles, not names. A large old-state update gate and a large candidate-write gate imply opposite mixing tendencies.

4.4 Why more than one memory route matters

The model can retain information in three KDA fields, the vector GRU, and the two-frame input history. It has no explicit top-down connection from final GRU state back to the encoder. The KDA states nevertheless affect subsequent visual processing through their emissions.

Suppose resetting all KDA matrices worsens accuracy. That establishes an effect of those states under that intervention, but it does not mean all useful memory resided exclusively in KDA. The GRU may depend on KDA's normal outputs and react badly when they are changed. Conversely, an apparently harmless reset may be masked by the raw-frame stack or redundancy in the GRU.

The final task head is linear. Complex spatial comparison must therefore already be encoded in h_T in a form the head can separate. There is no explicit hand-programmed comparator, object-slot matching mechanism, or experimenter-supplied target coordinate in this route.

4.5 Reading the parameter count correctly

The existing thirteen-head model has 2,750,324 trainable parameters: 256,992 in encoder blocks, 9,312 in KDA input projections, 74,262 in KDA modules, 2,007,296 in the dense feature projection, 394,752 in the vector GRU and 7,710 in task heads. These are static architecture counts, confirmed in the original report's source audit. [C1-C3; R1]

Most parameters are in the dense feature projection, while most trial-local recurrent scalars are in KDA. Parameter count measures learned degrees of freedom in the computation; activation-state size measures the amount of intermediate state carried for a particular example. They answer different questions.

5. ConvGRU and the planned spatial readout

5.1 What is actually being changed

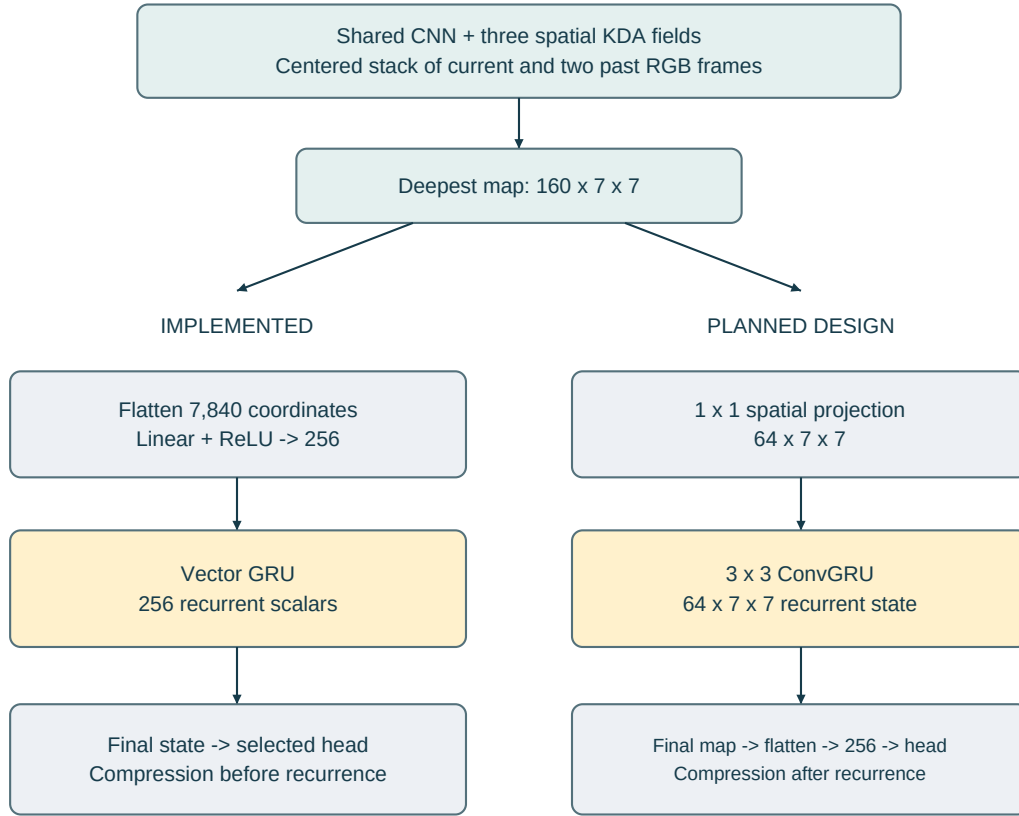
Design status: planned, under construction by another agent. The recorded proposal retains the centered three-frame stack, convolutional hierarchy, all three KDA modules and thirteen task heads. It replaces the final dense projection plus vector GRU with a spatial projection and a ConvGRU. The final dense projection moves to after recurrent processing. [D1]

The proposed route is

$$\begin{aligned} F_t &\in \mathbb{R}^{160 \times 7 \times 7} \xrightarrow{1 \times 1 \text{ conv}} U_t \in \mathbb{R}^{64 \times 7 \times 7}, \\ H_t &= \text{ConvGRU}(U_t, H_{t-1}), \quad H_t \in \mathbb{R}^{64 \times 7 \times 7}, \\ a_T &= \text{ReLU}(W_a \text{vec}(H_T) + b_a) \in \mathbb{R}^{256}, \quad \ell = W_{\text{task}} a_T + b_{\text{task}}. \end{aligned}$$

The essential change is **where spatial information is compressed**. The old route projects the deepest map to a 256-vector at every frame and then recurs. The proposed route recurs on a 7×7 grid and projects only the final spatial state. This provides the final recurrent computation with an explicit spatial organization throughout the trial.

It does not abolish compression. The 1×1 projection reduces 160 channels to 64, the recurrent field has finite capacity, and the final head still receives only 256 features. It also does not add an explicit target-selection controller or top-down feedback.



Gold = trial-local recurrent state. Both routes also contain KDA state upstream.

Figure 2: The planned branch moves the final spatial compression to after recurrent processing. It retains the same KDA hierarchy.

5.2 The ConvGRU equations

Convolutional recurrence uses shared spatial kernels to combine a current map and an earlier hidden map. Ballas and colleagues provide a primary precedent for ConvGRU video representations; our exact gate convention and dimensions come from the local design brief. [P2; D1]

Let $*$ denote convolution and $[U_t, H_{t-1}]$ concatenation along channels. The planned cell computes

$$\begin{aligned}
 [w_t, r_t] &= \sigma(K_g * [U_t, H_{t-1}] + b_g), \\
 \tilde{H}_t &= \tanh(K_c * [U_t, r_t \odot H_{t-1}] + b_c), \\
 H_t &= (1 - w_t) \odot H_{t-1} + w_t \odot \tilde{H}_t.
 \end{aligned}$$

The gate convolution has 128 input channels and 128 output channels, split into 64 write gates and 64 reset gates. The candidate convolution takes 128 channels and emits 64. Both use 3×3 kernels with padding that maintains 7×7 resolution. Hidden state starts at zero each trial; gate and candidate biases start at zero. Zero bias does not force every gate to 0.5 on arbitrary input, because the convolution weights still contribute. [D1]

A **write gate** close to zero carries the old hidden value forward; close to one replaces it with the candidate. A **reset gate** close to zero removes that old hidden component from the candidate's input. It does not directly erase the carried state: the carry term is governed by the write gate.

For example, let one hidden component be 0.8 and its candidate be -0.2 . With $w = 0.1$, the new value is 0.7. With $w = 0.9$, it is -0.1 . The same candidate can mean a minor revision or almost complete replacement depending on the write gate. This example isolates the final mixing operation; real candidates and gates depend on neighboring features.

5.3 What recurrence can do with space

A 3×3 convolution gives each new hidden location access to a local neighborhood of earlier hidden values and current inputs. Repeated steps can propagate information across more of the hidden map. Through the reset computation, the candidate also has indirect dependencies beyond a single simple neighborhood, so “one cell per frame” would be an oversimplified propagation law.

A network could learn to maintain an orientation representation at a location, bring a later probe representation into contact with it, or integrate evidence while preserving which region supplied it. These are possible computations, not guaranteed emergent behaviors. Kernel sharing favors reuse of local operations across positions. Padding, upstream features and the final position-specific dense readout mean that the whole model is not guaranteed to be perfectly translation invariant.

Starting from zero, the stated ConvGRU update keeps each hidden component within $[-1, 1]$ in exact arithmetic: the candidate is bounded by $\tanh$ and each update is a convex mixture with the earlier bounded component. This value bound does not establish nonexpansive gradients; the gates and candidate themselves depend on hidden state. It also does not directly apply to KDA matrices, whose values are signed and unconstrained.

5.4 KDA and ConvGRU store different things

Question	Spatial KDA	Planned final ConvGRU
Persistent state	Matrix per site and head	Hidden channel vector per site
Write rule	Correct a key-addressed prediction error	Mix old state with a nonlinear candidate
Retention control	Decay along key rows	Component-wise carry/write mixing
Read rule	Query the matrix, then project	Expose hidden map to final readout
Direct recurrent spatial mixing	No neighbor-state term in local matrix update	Neighboring hidden values enter convolutions
Proposed role here	Memory inside three encoder scales	Spatial recurrence after the deepest map

The new branch is a **KDA-plus-ConvGRU model**, not a contest in which ConvGRU replaces KDA. The two forms of state may learn complementary functions, redundant functions, or functions that interfere. Their coexistence does not establish a clean biological division into sensory memory and working memory.

5.5 Three architectures that must not be confused

The **existing joint model** has KDA at three scales and a final vector GRU. The **historical accumulator comparison** could put a 32-channel ConvGRU at those three scales, in place of KDA, while retaining a final vector GRU. The **new planned branch** keeps the KDA hierarchy and adds a 64-channel final ConvGRU in place of the vector-recurrent route. [C1-C2; D1]

A performance result from the historical ConvGRU accumulator arm does not measure the new branch. It differs in component placement, state dimensions, training history and task exposure. Its existence helps explain the design vocabulary, but it cannot stand in for the pending implementation or its future results.

5.6 What the new experiment could teach us

The proposed final hidden field has $64 \times 7 \times 7 = 3,136$ scalars per example, compared with 256 in the old final GRU. This is 12.25 times as many final recurrent scalars; it is not a 12.25-fold increase in the whole model's state, because the 215,808 KDA scalars remain. The proposed combined count is 218,944, excluding buffers and training intermediates.

The design warm-starts compatible CNN, KDA and task-head parameters from a recorded global-GRU checkpoint, while initializing the new spatial projection, ConvGRU and final projection. All learned parameters remain trainable. Compatible optimizer state is to be migrated by parameter name and shape. This is a planned training design, not a completion receipt. [D1]

Preserving the task heads does not preserve their input representation: their 256-dimensional inputs will now come from a newly initialized route. Some initial disruption is therefore plausible. Later improvement would show that this new branch can learn useful behavior under its exposure. Without a matched continuing control, improvement would not isolate the benefit of spatial recurrence from extra training, changed capacity, initialization or other changes in the readout.

6. The seven two-frame sensory tasks

6.1 What this group isolates

These tasks ask whether the system can extract and compare visual information before we demand extended retention or complex spatial instructions. Each trial contains exactly two presented images and one final supervised answer. There is one primary condition, mixed, per task; difficulty values are mixed within that condition. Different difficulty values are reporting strata, not additional primary cells. [C4-C6]

All seven tasks still use the recurrent architecture. However, the final three-frame stack contains both observed images. Good performance here does not establish long-term retention. These tasks are useful prerequisites and controls: weak elementary motion makes a failure on motion-duration accumulation hard to attribute specifically to memory.

A binary label is not universally “change.” Five sensory tasks ask **which interval** contains a particular property. Signed orientation asks **which sign** the rotation has. Motion asks **which direction**. These meanings must remain attached to the corresponding task head.

6.2 Motion direction: correspondence across two images

Task key: motion_direction. **Labels:** right = 0, up = 1, left = 2, down = 3. **Chance:** 25% for balanced four-way decisions.

Two random-dot images depict one of four cardinal displacement directions. There are 256 dot identities in a periodic square domain; a circular aperture determines which rendered dots are visible. Half the identities survive between images and move in the chosen direction. The other half are regenerated independently. The displacement is one, two or three pixels. Thus “256 dots” does not mean all 256 are simultaneously visible inside the aperture. [C5]

Example. If surviving dots move two pixels to the left, the answer is class 2. Randomly replaced dots provide distractor correspondences. A model needs a statistic of ordered spatial correspondence, not the absolute location of one bright dot. Averaging the two frames symmetrically would discard the sign of motion.

The useful invariant is the displacement direction despite random initial positions and replacements. The renderer gives no intended direction cue in the distribution of an individual frame. Nevertheless, verifying that a trained model relies on the pair requires an actual single-frame or order-control evaluation; it does not follow just from naming the task.

What success teaches us. The model can read short-range motion information in this raster regime. It does not establish accumulation over eight transitions, selection among four patches, or change detection between noncardinal directions. Those are distinct later tasks.

6.3 Signed orientation: compare axes, not pixel phases

Task key: orientation. **Labels:** negative axial-angle change = 0, positive change = 1.

A localized grating appears in each frame. The base orientation varies, and the second orientation differs by ± 4 , ± 10 or ± 22 degrees. The phases of the two gratings vary independently. Phase changes move light and dark stripes without changing their orientation, so simple pixel subtraction is not a sufficient orientation representation. [C5]

Grating orientation is *axial*: rotating by 180° gives the same axis. A convenient signed comparison is

$$\Delta\theta = \frac{1}{2} \operatorname{atan2}(\sin 2(\theta_2 - \theta_1), \cos 2(\theta_2 - \theta_1)).$$

For example, 175° followed by 5° can represent a positive 10° axial change rather than a negative 170° change. This formula explains the geometry; the model is not given the numerical angles or this comparator.

The repository's older prose uses inconsistent clockwise/counterclockwise wording. Use the renderer's numerical sign rather than redefining labels from screen-coordinate intuition. **Every trial has a nonzero change.** The task is not change versus no change. High accuracy would establish signed comparison at these sampled magnitudes, not a measured orientation threshold below four degrees.

6.4 Contrast: identify the stronger modulation

Task key: contrast. **Labels:** first frame = 0, second frame = 1.

The two frames share a grating pattern. One has a baseline contrast, or pedestal, of 0.08, 0.18 or 0.30; the other adds 0.025, 0.06 or 0.13. The correct interval is the one with the larger modulation around the background level. This is not simply which frame is brighter on average. [C5]

Example. Frame 1 has contrast 0.18 and frame 2 has 0.24. The increment is 0.06 and the correct label is 1. The same increment from a pedestal of 0.30 creates a smaller relative change. Report both pedestal and increment so a high mixed score does not conceal weakness at subtle relative differences.

A plausible computation estimates orientation-insensitive modulation energy and compares it across intervals. The network is not required to implement that particular estimator. Strong contrast discrimination demonstrates access to this image statistic; it does not by itself demonstrate spatial attention or object recognition.

This task also illustrates a limitation of architectural intuition. Normalization can alter contrast information, but one cannot infer complete contrast removal from the presence of GroupNorm: convolution biases, nonlinearities, spatial structure and multiple channels can retain usable cues. The learned system's behavior settles whether the available route is sufficient.

6.5 Spatial frequency: compare stripe density

Task key: spatial_frequency. **Labels:** first frame = 0, second frame = 1.

The higher-frequency grating has more cycles over the same image extent. The baseline is sampled between four and nine cycles per image, and the higher interval multiplies it by 2^δ , where δ is 0.08, 0.18 or 0.35 octaves. Independent phases prevent the decision from reducing to a fixed stripe alignment. [C5]

Example. With a six-cycle baseline and an increment of 0.18 octaves, the higher frequency is approximately $6 \times 2^{0.18} = 6.80$ cycles per image. If it appears first, the label is 0. An octave is a frequency ratio, so the absolute difference depends on the baseline.

The network must distinguish orientation, phase and frequency rather than confusing any pattern difference with the requested property. Convolutional channels with different effective frequency sensitivity could support the comparison, but the code contains no hand-built frequency classifier.

Success establishes discrimination in this sampled range and rasterization. It is not scene recognition, and it is not proof of human-like frequency tuning. A useful performance report separates all three increments rather than treating the mixed average as one universal frequency threshold.

6.6 Chromatic increment: a specified colour direction

Task key: chromatic_increment. **Labels:** first frame = 0, second frame = 1.

A soft-edged patch appears on gray. One interval adds an increment of 0.018, 0.045 or 0.1 along a specified linear-RGB colour axis. Let $w = (0.2126, 0.7152, 0.0722)$ be the renderer's luminance weights. Its colour direction is proportional to

$$a = (0.7152, -0.2126, 0), \quad w^\top a = 0.$$

After normalizing a , the target colour is $c + \delta a$. Under this numerical luminance definition, the increment changes colour without changing the patch's nominal luminance. Small image noise remains. This is a renderer-defined colour discrimination, not a calibrated claim about a particular display or human isoluminance. [C5]

Example. If the second patch has the positive increment, its label is 1 even if a viewer informally describes the difference as “redder” or “less green.” The label is the specified axis increment, not an arbitrary colour preference.

Because the baseline RGB colour varies, learning a fixed absolute target colour is insufficient as a general solution. The model should compare the intervals along the relevant direction. Strong performance supports sensitivity to the defined colour contrast; it does not imply categorical colour naming, constancy under illumination changes, or spatial selection.

6.7 Contour grouping: the arrangement is the signal

Task key: contour. **Labels:** first frame = 0, second frame = 1.

Both images contain 32 small Gabor elements. In the structured interval, seven are arranged and oriented along a curved path. Their alignment jitter is two, eight or sixteen degrees. The control interval permutes the orientation assignments across the same positions. The two images therefore share the orientation multiset, but differ in which orientations occur at which locations. [C5]

Example. One image contains seven appropriately aligned elements along an arc; the other redistributes those orientations among the 32 positions. If the aligned arc is in the first image, the answer is 0. A histogram of orientation counts alone cannot distinguish the intended manipulation because the multiset is matched.

The challenge is spatial organization: local orientation must be related to neighboring positions and the path geometry. A network can exploit local alignment statistics without developing a human-like percept of a completed object boundary. That distinction is useful when interpreting success.

Larger jitter weakens path alignment. Reporting only an overall accuracy can hide whether the system handles the highly aligned case but fails when the same path becomes less regular. A low score might reflect insufficient spatial grouping, inadequate receptive-field use, difficult optimization or weak local features; it is not directly a working-memory capacity result.

6.8 Natural spectral detail: a photograph task without identity memory

Task key: `natural_spectrum`. **Labels:** first frame = 0, second frame = 1.

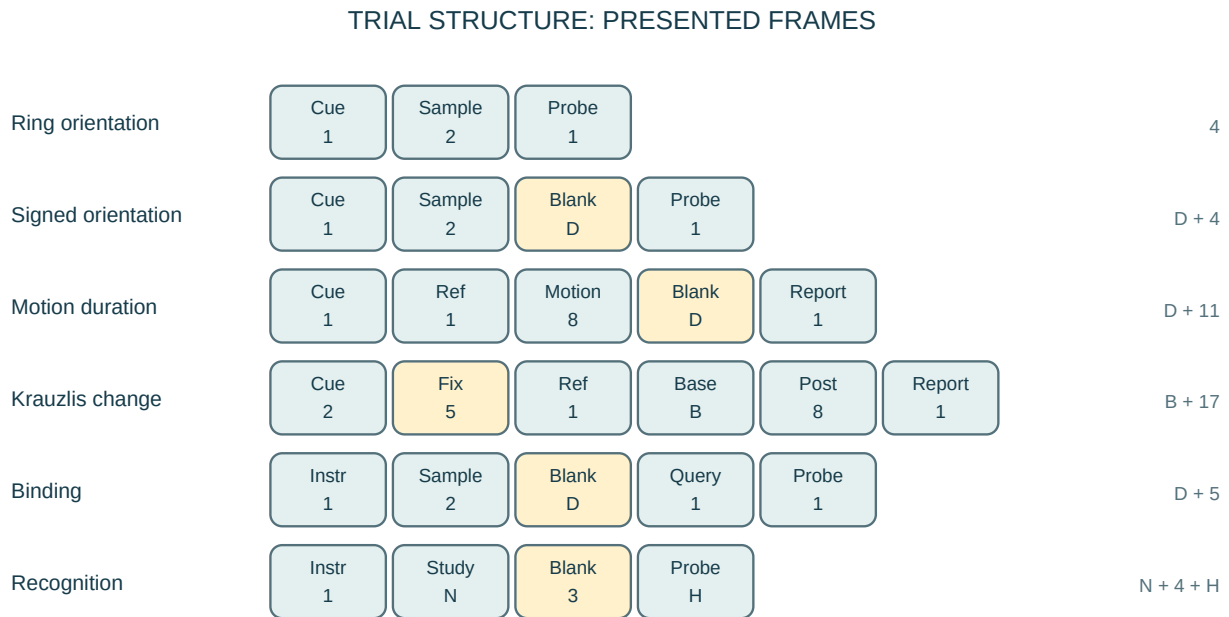
Both intervals derive from the same natural photograph crop. The renderer converts to luminance, preserves Fourier phase, and changes the relative frequency content. Its multiplier is proportional to $(r/0.1)^{-\beta}$ at radial frequency r , with DC treated separately. A smaller β retains more high-frequency detail relative to low-frequency structure. The two beta values differ by 0.15, 0.30 or 0.60. The resulting images share mean and RMS contrast. [C6]

Example. If the first interval uses the smaller beta, it is the correct answer even though both images show the same scene. “More detail” here has an exact spectral meaning, rather than a judgment about semantic informativeness.

Matching mean and RMS prevents the intended discrimination from being a trivial comparison of overall brightness or contrast energy. Phase preservation maintains much of the scene’s spatial structure while redistributing spectral energy. The model can succeed by reading texture or edge-scale statistics; it need not name the scene or remember its identity across a list.

The BSDS500 photograph identities are split into train, validation and test pools. These same identity boundaries are shared with scene recognition. Generalization to held-out photographs is therefore different from sampling another random crop of a photograph already used in training. This task and recognition should never be merged into one vague “natural-image ability.”

7. Six tasks with spatial instructions or longer sequences



Schematic block widths are not durations. Numbers in blocks are frame counts.

Figure 3: The six longer task formats differ in cue timing, evidence, and report. D denotes blanks, B baseline motion, N study load, and H probe repetitions.

7.1 Ring-cued orientation: location plus signed change

Task key: orientation_ring. **Labels:** negative target rotation = 0, positive target rotation = 1. **Primary cell:** D0.

The trial is **one cue frame, two sample frames, one probe frame**. Four Gabor patches occupy the established four locations. A ring marks the target during the cue and sample frames. Each patch rotates between sample and probe; the foils have independently chosen signs. Rotation magnitudes are 15, 30 or 45 degrees. There are no inserted retention blanks in this suite cell. [C7]

Example. The ring selects the upper-right patch. That patch rotates by +30°, while two foils rotate negatively and another rotates positively. The answer is 1 because the target's sign is positive. A majority vote over all patch signs is not the rule.

This combines location selection with signed orientation comparison. It does not ask the model to interpret a plus/minus instruction, and it contains no unchanged target negatives. Those distinctions make it different from orientation_cued, despite both having binary outputs.

The final input stack includes the two samples and probe together. A good solution can therefore be largely a spatially selective image comparison at the final step. This is why historical ring-first training could serve as an acquisition rung, but ring success alone cannot establish long-delay memory. In the current joint suite, it is one task among thirteen; its inclusion does not impose a ring-first curriculum.

7.2 Signed cued orientation: apply a rule to the chosen location

Task key: orientation_cued. **Labels:** target rotation agrees with instruction = 1; unchanged or opposite-sign target rotation = 0. **Cells:** D0, D4, D12, D24.

The trial is **cue 1, sample 2, blank delay D , probe 1**, totaling $D + 4$ frames. A plus/minus glyph near one location simultaneously identifies the target and the sign to report. It is visible in the cue and both samples, then absent from the probe. The four centers are (27, 27), (73, 27), (27, 73) and (73, 73) in image coordinates. [C8]

Let $s \in \{-1, +1\}$ be the glyph's sign and $\Delta\theta_j$ the target's signed rotation. The label rule is

$$y = \mathbf{1}[s\Delta\theta_j > 0].$$

Examples. A minus instruction with a target rotation of -15° is positive. The same instruction with $+15^\circ$ is negative. A zero rotation is also negative. Thus a real physical target change can correctly receive a negative report.

The renderer constructs a sign-relative rotation multiset before assigning the target, preserving aligned, opposite and unchanged locations on every trial. This makes a global inventory of change signs insufficient: the network needs the relation among instruction, target location and target rotation.

A successful computation could retain target orientation and the requested sign until the probe. It could instead keep a richer scene and select later. The architecture does not stipulate the strategy. Failures can arise from glyph recognition, location association, orientation precision, retention or final rule application. Looking at all four delays and comparing with the ring task helps organize these possibilities, but does not uniquely diagnose them.

7.3 Cued motion duration: count evidence across transitions

Task key: motion_duration_cued. **Labels:** right/up/left/down = 0/1/2/3. **Cells:** the same four delays.

The trial is **cue 1, reference 1, moving frames 8, blanks D , report 1**, totaling $D + 11$ frames. Four dot patches have independently generated direction schedules. A ring selects one patch and remains visible during the reference and moving frames. Each patch has 32 dots; on every transition, sixteen randomly chosen dots and any boundary-crossing dots are replaced. Step size is 0.8, 1.2 or 1.6 pixels. [C8]

The correct answer is the unique direction with the largest number of the target's eight transitions:

$$y = \arg \max_{d \in \{0,1,2,3\}} \sum_{t=1}^8 \mathbf{1}[d_t = d].$$

This formula defines the ground truth. The network receives images, not the direction sequence or a counter.

Example. Consider right, up, right, left, down, right, up, left. Counts are (3, 2, 2, 1), so the answer is right, even though the last transition is left. The corresponding net displacement is not a categorical count of the most frequent direction either. A strategy based on final motion, one salient jump, or pooled evidence from all patches can fail.

The task demands elementary motion estimation, target selection, temporal accumulation and retention until report. The report frame supplies no new motion evidence. Strong two-frame motion performance is therefore a useful prerequisite but not a solution to this task.

A useful diagnostic separates count-winner margin from recency. A narrow margin is intrinsically less tolerant of one misread transition; a strong dependence on the last transition may indicate recency weighting. Neither observation alone identifies whether the source is encoding, recurrence or the final readout.

7.4 Krauzlis-style motion change: reject a real foil event

Task key: `krauzlis_cued_motion`. **Labels:** change at the cued patch = 1; foil-only change or catch = 0. **Cells:** B12, B20, B28.

There are two patches, centered at (20, 50) and (80, 50). The trial contains **cue 2, fixation-only 5, reference 1, baseline transitions B , postevent transitions 8, report 1**, totaling $B + 17$ frames. The B values are baseline-motion lengths, not blank-memory delays. The renderer defines a 100 Hz clock; its timing is a compressed adaptation of Arcizet and Krauzlis's task, not a faithful recreation of the entire experiment. [C8; P4]

The patches begin with mean directions separated by 90° . An event rotates one mean by $\pm 26^\circ$ or $\pm 28^\circ$, or produces no change. Each patch has sixteen dots, subpixel bilinear rendering, 0.375-pixel steps, directional variability and finite dot lifetimes. Motion is not restricted to four cardinal directions.

Over a complete 100-trial event cycle there are 57 target events, 29 foil events and 14 catches. **Example:** cue left, change right means label 0, although a genuine motion change occurred. Cue left, change left means label 1. No change means label 0 regardless of cue side.

An always-positive model would achieve 57% raw accuracy on that exact mixture, 100% target hits, and 100% foil and catch false reports. Its balanced accuracy would be 50%. Hits alone would therefore make a failed selective observer look excellent.

This task teaches the difference between detecting change and selecting relevant change. Report target hits, foil false alarms and catch false positives separately. A foil error may reflect poor spatial selection; a catch error may reflect an overly liberal criterion or noisy change estimation. Those are useful hypotheses, not automatic mechanistic diagnoses. The model outputs one final decision, so it does not produce a learned reaction-time distribution.

7.5 Spatial binding: remember which feature belonged where

Task key: `spatial_binding`. **Labels:** the queried item participated in the exchange = 1; only foils exchanged = 0. **Cells:** four delays.

The trial is **instruction 1, sample 2, blanks D , retrospective ring query 1, probe 1**, totaling $D + 5$ frames. No target is marked during sample encoding. The four sample orientations come from an inventory spaced 45° apart around an axial circle with a random offset. At probe, exactly one pair of locations exchanges orientations. The overall inventory stays the same. [C8]

Example. Suppose the four locations contain $[10, 55, 100, 145]^\circ$. After the delay, the query selects the first location. Probe $[55, 10, 100, 145]^\circ$ is positive because the queried item exchanged with the second. Probe $[10, 100, 55, 145]^\circ$ is negative because only the second and third exchanged.

Both trials contain exactly two changed locations and exactly the same set of orientation values as their sample. Neither an "anything changed" detector nor an unordered bag of remembered orientations solves the required distinction. The model needs information about the association between orientation and location, accessible after the retrospective query.

The late cue changes the informational requirement. The model cannot know in advance which location will matter from an encoding precue. It must preserve useful information across the initially eligible locations, or adopt another strategy that still supports the eventual query. Calling this ordinary precued selective storage would misdescribe the trial.

Successful performance supports usable feature-location information for these four familiar centers and sampled delays. It does not directly estimate an unrestricted object capacity or generalization to new spatial layouts. A query failure and a lost binding can both cause an incorrect answer, so diagnostic probes must distinguish availability from use.

7.6 Scene recognition: exact membership after a list

Task key: image_recognition. **Labels:** exact study-list member = 1, nonmember = 0. **Cells:** every combination of study load $N \in \{0, 4, 12, 24\}$ and probe hold $H \in \{3, 4, 5\}$, giving twelve cells.

The trial is **instruction 1**, N **distinct study images**, **blanks 3**, H **copies of one probe**, totaling $N + 4 + H$ frames. Each study image appears for one frame. Photographs are canonically cropped and resized to 100×100 RGB. Positive probes are byte-identical copies of a study raster; negatives are distinct from all study rasters. No instruction glyph overlays the photograph frames. [C8]

Example. After studying $[A, B, C, D]$, the repeated probe C, C, C, C is positive. A repeated new image E, E, E, E is negative. The repetition holds the same probe on screen; it does not create four independent tests or four new studied items.

The third blank removes direct study-image access from the three-frame stack before probe presentation. The model must retain some useful list information elsewhere. It need not reconstruct every image. A learned compressed familiarity or matching representation could suffice, provided it generalizes to the held-out photograph pool and respects membership.

For $N = 0$, every trial is negative. Report specificity and false-positive rate; balanced accuracy and AUC are undefined because one true class is missing. These three empty-list cells cannot raise a claim about recognition of nonempty lists. For nonempty cells, compare load and probe hold separately. A longer hold permits more recurrent computation on the probe, but does not increase study exposure.

A subtle challenge is that a negative probe becomes familiar during its own repetitions. The correct label must remain negative: membership refers to the study list, not to whether the probe has appeared anywhere earlier in the ongoing trial. The recurrent computation must therefore distinguish study evidence from probe repetition, rather than indiscriminately interpreting every recent match as a positive.

The task tests exact-raster membership. It does not require recognizing the same scene from a new viewpoint, a different crop or altered illumination. It is also not the spectral-detail task: recognition compares the probe with a retained list, whereas spectral detail compares two transformed views of the same source crop.

7.7 The complete condition map

Task group	Tasks	Cells per task	Total cells
Two-frame sensory	7	1	7
Ring orientation	1	1	1
Signed cued orientation	1	4	4
Cued motion duration	1	4	4
Krauzlis motion change	1	3	3
Spatial binding	1	4	4
Scene recognition	1	12	12
Total	13		35

Thirty-two cells have both-class or multiclass scoring eligibility. The three empty-list recognition cells remain visible but are excluded from BA/AUC checkpoint selection. Difficulty strata, such as rotation magnitude or motion displacement, sit within these cells rather than changing the 35-cell total. [C4]

8. How the tasks train a shared recurrent system

8.1 One final loss, many earlier computations

Each trial produces logits from the selected task head. Cross-entropy penalizes low probability on the correct class:

$$L = -\frac{1}{B} \sum_{b=1}^B \log p_{b,y_b}.$$

Only the selected head participates in that task’s computation. The shared encoder, KDA modules and final recurrent route can receive gradients from every task. “All learned parameters are trainable” means no component is deliberately frozen; it does not mean every parameter receives a nonzero gradient on every batch. Inactive task heads have no gradient for that update. [C3]

There are no supervised target-angle labels for the internal state, no gate target saying “retain now,” and no extra output at every frame in the deployed model. Full-sequence backpropagation propagates the final decision’s loss through the intervening state transitions. It can teach an early gate to preserve information if doing so improves the eventual answer. This is why task design and temporal gradient flow matter even when the visible supervision is only a final class label.

For a generic recurrence $z_t = f_\theta(z_{t-1}, x_t)$, credit to an early state involves products of state-transition Jacobians. Long paths can attenuate or amplify gradients. Gating can provide useful carry paths, but does not guarantee successful credit assignment. The local KDA nonexpansion result in Chapter 2 is not a blanket proof about this full training computation.

8.2 Effective batch size and microbatches

The existing joint trainer uses effective batch size 32 through eight microbatches of four. Each microbatch computes mean cross-entropy, scales it by $4/32$, and accumulates gradients before one Adam step. All eight microbatches belong to the same chosen task and condition. [C3]

This reduces peak batch memory while preserving the intended average gradient, up to numerical differences, for this model’s per-example computation. It does not truncate a sequence: each microbatch backpropagates through its complete trial. Microbatching across examples and truncating through time are different operations.

The recorded optimizer is Adam with learning rate 10^{-4} , betas (0.9, 0.999), epsilon 10^{-8} and zero weight decay. There is one parameter group, fp32 computation, finite-value checks, and no gradient clipping. These describe this training recipe, not a claim that it is universally optimal. The new branch’s brief preserves these settings while changing the final architecture. [C3; D1]

8.3 Equal tasks do not mean equal conditions or frames

A shuffled cycle contains one update from each of thirteen tasks. Within a task, a shuffled queue cycles through its conditions. Thus task exposure is balanced by optimizer updates, while a twelve-cell task receives fewer updates per individual cell than a one-cell task over the same total horizon. [C3]

Over 156 updates, there are twelve updates per task. A one-cell sensory task receives all twelve in its single cell. A four-cell task receives three per cell. Recognition receives one per cell. With 32 examples per update, each task receives 384 episodes over this illustration, but its allocation across conditions differs.

Episodes also differ in length. A two-frame task uses far fewer image frames than a long motion or recognition trial. Equal episodes therefore do not imply equal frame exposure or equal compute cost. When interpreting learning speed, retain three counts: optimizer updates, independent episodes, and processed frames.

Joint training creates a shared representational problem. A feature useful for contrast might be irrelevant to binding; a gate useful for sensory comparison may need a different pattern for retention. Loss curves alone cannot establish destructive gradient interference. That hypothesis needs a comparison or diagnostic designed to measure it.

8.4 Acquisition, validation and testing

Training changes weights. Validation selects among checkpoints or checks acquisition under a declared policy. Final testing evaluates the selected result on a separate split. Repeatedly inspecting a fixed validation set does not make each look independent new evidence.

The suite maintains separate task/condition streams. Sampling one condition does not advance another. Its ordinary delay cells are independent draws, not the same stimulus rendered at multiple delays. To estimate the causal effect of delay on a matched scene, a separate paired evaluation must hold the base stimulus fixed and change only the delay. [C4]

The existing project's selection policy changed between training phases. The first short run used a worst-task chance-normalized BA criterion; later continuation used equal-task mean AUC with a BA tie-break. Those are different optimization preferences. A minimum rewards the weakest task but can be sensitive to a noisy floor; a mean can improve while difficult tasks remain at chance. Neither criterion substitutes for inspecting the complete task vector. [R1; C3]

The supplied note's step-715 test and step-2314 validation are historical snapshots with different evaluation roles. They should not be narrated as a paired architectural comparison or as the latest model state. This book uses no new performance measurements.

8.5 Accuracy, balanced accuracy and AUC answer different questions

Raw accuracy counts correct decisions. Balanced accuracy averages recall across the true classes:

$$\text{BA} = \frac{1}{K} \sum_{c=1}^K \frac{\text{correct predictions in class } c}{\text{examples in class } c}.$$

For binary tasks with both classes represented, a constant-class predictor has BA 0.5, even when the class frequencies are unequal. For the four-direction tasks, balanced chance is 0.25. A useful cross-task scaling is

$$\text{BA}_{\text{normalized}} = \frac{\text{BA} - 1/K}{1 - 1/K}.$$

This maps chance to zero and perfect performance to one. It does not equalize uncertainty or task difficulty. [C3]

Binary AUC measures ranking: the probability that a randomly chosen positive receives a higher positive score than a randomly chosen negative, with a half-credit convention for ties. Consider negatives scored 0.60, 0.61 and positives 0.70, 0.71. At a threshold of 0.5, every example is called positive, so BA is 0.5. Yet AUC is 1 because the ordering is perfect. This constructed example explains how a ranking signal can coexist with a useless deployed threshold.

The project's multiclass AUC averages one-versus-rest AUCs. Its chance reference is 0.5, not 0.25. AUC is not accuracy, and finding a promising ranking does not itself demonstrate a corrected decision rule. Threshold fitting must have its own training/validation separation.

8.6 What should be in a learning report

For each task, keep the primary conditions, denominators, confusion matrices and relevant difficulty strata visible. Preserve the special Krauzlis event decomposition and empty-list recognition specificity. Average eligible conditions within a task before computing an equal-task summary, so recognition does not dominate merely because it has twelve cells.

A trained seed is one trained realization. More test trials reduce uncertainty about that realization's behavior on the stimulus distribution; they do not replace independent training seeds. Photograph-based uncertainty also needs to account for repeated use of a finite source pool. A visually impressive average is useful only when the underlying unit of evidence is clear.

9. From successful behavior to a mechanism

9.1 Availability and use are different questions

Suppose a frozen-state decoder predicts sample orientation accurately just before the probe, but the model's final answer is poor. This would show that the extracted state contains decodable information for that decoder and evaluation distribution. It would motivate investigating access, comparison or report generation. It would not prove that the deployed readout can use that information in the same way.

Conversely, a failed linear probe does not prove erasure. The code may be nonlinear, distributed across states, or inaccessible to the chosen analysis. Probe capacity, training data, target definition and evaluation split all matter. For axial orientation, a decoder should respect the doubled-angle geometry rather than treating 1° and 179° as maximally distant.

A useful diagnosis follows the computation: test sensory representation, target/instruction use, retained content, comparison and output. This order prevents the word “memory” from becoming a catch-all explanation for every sequence-task failure.

9.2 Three senses of attention

Associative attention is query-dependent retrieval from a matrix. **Spatial selection** is behavior that depends on the instructed location rather than irrelevant alternatives. **Competitive spatial allocation** is a mechanism that explicitly normalizes influence across locations, such as a spatial softmax.

Our KDA implements the first and can support learning the second. It does not contain the third as an explicit spatial softmax. Its α is a row-retention gate, not a normalized probability of attending to an image location. Calling every α map “attention allocation” would collapse different mechanisms into one word.

The recurrent vision transformer studied by Morgan, Albanna and Herman uses memory-guided spatial attention and a learned wait/declare-change policy. Its cue-validity manipulations and attention-bias interventions concern a different computational substrate from our local associative matrices and fixed final report. Its findings motivate comparison, but not an assumption of equivalent dynamics. [P5]

9.3 Perturbing an emission versus perturbing memory

The original note proposes computational interventions; it does not report them as completed on the joint learner. An **emission pulse** adds a spatially localized vector to O_t after the KDA output projection. This changes what downstream processing receives at that frame without directly overwriting that module's saved matrix. It can nevertheless have lasting effects through higher-scale KDA and final recurrent state.

A **state pulse** instead changes the carried matrix. For a rank-one perturbation $\Delta S = \lambda ab^\top$, the immediate raw read changes by

$$\Delta o = \Delta S^\top q = \lambda(a^\top q)b.$$

If a is orthogonal to q , the immediate raw read is unchanged even though memory was altered. A later query may expose the change. This is why perturbation size alone is not an adequate manipulation check.

These are interventions on mathematical representations. Their dose has no established conversion to electrode current, recruited neurons or tissue extent. Signed KDA coordinates also make “add a positive number to every channel” an ambiguous analogue of physiological excitation. A direction should be calibrated and its achieved effect measured. [R1]

9.4 What a controlled causal comparison needs

Compare identical episodes under sham and perturbation, with a frozen checkpoint and a verified analysis wrapper. Specify the altered substrate, scale, location, temporal phase and dose. Distinguish sample encoding from the first two nominal blanks, which retain sample pixels in the raw stack, and from later genuinely sample-free processing.

Compare target, foil and off-target sites, and match the intervention geometry and magnitude as far as the representation allows. Include patterns that test whether an effect is specific to the chosen feature direction rather than generic disruption. This is an experimental design lesson, not authorization to run an intervention.

Behavioral interpretation must respect the label. Increasing reports of an uncued change is a false-alarm increase in the Krauzlis task. An opposite-sign orientation change is negative under an incompatible sign instruction. More positive reports are not automatically better perception.

For a binary task, separate hits and false alarms. Where a signal-detection model is appropriate,

$$d' = \Phi^{-1}(H) - \Phi^{-1}(F), \quad c = -\frac{1}{2}[\Phi^{-1}(H) + \Phi^{-1}(F)].$$

Sensitivity d' and criterion c distinguish discrimination from reporting tendency under the model's assumptions. Rates of zero or one require a declared finite-sample correction. Four-choice direction judgments should primarily use multiclass confusion, not an unexplained binary conversion.

A null perturbation effect also needs interpretation. The manipulation may have missed the relevant representation, been hidden from the current query, or been compensated by another route. Necessity, sufficiency, decodability and normal use are different claims. Well-designed tests make those distinctions visible rather than turning every outcome into confirmation.

10. Exercises with worked answers

10.1 Work the associative update

Question. Begin with zero state, a unit key $k = (1, 0)^\top$, value $v = (3, 2)^\top$, query equal to the key, retention one and write strength 0.5. Present the same key/value twice. What are the two reads?

Answer. The first prediction is zero. The first write makes the first row $(1.5, 1)$, which is also the first read. At the second step, the error is $(1.5, 1)$ and half of it is added. The second read is $(2.25, 1.5)$. Repeated writes approach $(3, 2)$ rather than adding it without bound. Retention one is an idealized limiting case used to isolate the delta correction.

10.2 Find the interference

Question. A write has $\beta = 0.5$, unit key $k = (1, 0)^\top$ and error $e = (2, -4)^\top$. What changes for query $q' = (0, 1)^\top$? What about $q'' = (1, 1)^\top / \sqrt{2}$?

Answer. The orthogonal query changes by zero. The diagonal query changes by $0.5(1/\sqrt{2})(2, -4)^\top = (1/\sqrt{2}, -\sqrt{2})^\top$. The same write has different effects depending on address overlap; no spatial change is needed for interference within a site's associative code.

10.3 Read a gate correctly

Question. A proposed ConvGRU cell has write gate near zero and reset gate near zero. Does reset erase its old state?

Answer. No. Reset removes old-state input to the candidate computation, but the near-zero write gate preserves the old state in the final mixture. If the write gate were near one, the state would instead be replaced by the largely current-input-driven candidate.

10.4 Identify the changed architectural hypothesis

Question. The new branch improves binding. Can we conclude that replacing KDA with ConvGRU improved binding?

Answer. No. The new branch keeps KDA and changes the final recurrent readout. Moreover, its warm start and additional exposure mean that improvement without a matched control would not isolate the architecture's causal contribution. The defensible observation would be that the new combined branch acquired better binding under its measured training history.

10.5 Interpret a signed cue

Question. A minus glyph selects the lower-left patch. Its rotation is positive, while an uncued patch rotates negatively. Is the correct report positive?

Answer. No. The rule applies to the selected patch. Its rotation disagrees with the requested sign, so the label is zero. Selecting a foil with the desired rotation is an error even though the desired motion exists somewhere in the image.

10.6 Separate binding from global change

Question. In a binding negative trial, did nothing change?

Answer. Two foil locations exchanged orientations. The queried location was preserved. Every trial contains a swap, and the orientation inventory is unchanged. Global change detection or an unordered orientation inventory cannot provide the required answer.

10.7 Audit an apparently impressive score

Question. A Krauzlis model reports “change” on every trial and is described as achieving perfect target detection. What is missing?

Answer. Its foil and catch false-positive rates are also 100%. On the prescribed event mixture, raw accuracy is 57% and BA is 50%. Perfect hit rate here is compatible with complete failure to discriminate or select.

10.8 Count evidence, not repeated frames

Question. A recognition trial has $N = 12$, $H = 4$. How many frames and independent episodes does it contain? What changes if H becomes five?

Answer. It has $12 + 4 + 4 = 20$ frames and one episode. Increasing H to five creates 21 frames in the episode, not another independent membership judgment. It permits an additional recurrent update on the same probe.

10.9 Distinguish retained information from successful behavior

Question. A strong pre-probe decoder reads the sample orientation from KDA, but the deployed model answers at chance. What should the next explanation focus on?

Answer. The result makes complete absence of decodable sample information less plausible for the tested state and decoder. It motivates testing whether the normal query, final recurrent route and head access and use that information, and whether the correct location and instruction are represented. It does not by itself prove which of those stages failed.

Reference sheet and source map

Notation at a glance

Symbol	Meaning in this book
B, T	Batch size and presented sequence length
D	Inserted blank frames in delay tasks
B in B12, B20, B28	Baseline transitions, not batch size in those cell names
N, H in recognition	Study-list load and repeated-probe hold
S_t	KDA key-by-value matrix at a site/head
q_t, k_t, v_t	Query, key and incoming value
α_t, β_t	Row retention and scalar write strength
O_t, F_t	KDA emission and deepest combined visual map
h_t, H_t	Vector GRU state and spatial ConvGRU state
w_t, r_t	ConvGRU candidate-write and reset gates
z_t	Old-state mixing gate in PyTorch's vector GRU

Implementation and design sources

Paths below are relative to the supplied **VisualAttentionWorkingMemory** repository. They are references for readers, not instructions to execute the historical plans they contain. A separate `source_manifest.json` records SHA-256 identities of the local sources used for this edition.

C1 - Associative update and historical accumulator cells. `PreAttentiveVision/TemporalIntegration/accumulators.py`: `kda_update`, `SpatialKDA`, `SpatialConvGRU`. Defines the local equations, packed channel layout, normalization, gate initialization and historical 32-channel ConvGRU. Chapters 2-3 derive the code's algebra rather than relying on a language-model paper for implementation-specific claims.

C2 - Existing hierarchy and readout. `WorkingMemory/PlainBaseline/accum.py`: `AccumulatorBaseline`, `frames`, `encode_frame`, `forward`. Defines convolution widths, concatenation, centered stacking, dense projection, vector GRU and selected head.

C3 - Existing joint learning. `SecondPass/JointTraining/worker.py` and `core.py`: `model/optimizer` construction, `microbatch` accumulation, `scheduler`, `checkpoint` state and `scoring`. The continuation policy is documented separately in `continuation_v3.py` and `AMENDMENT_V3.md`.

C4 - Complete task inventory. `SecondPass/TaskSuite/README.md`, `catalog.json` and `suite.py`. Defines all thirteen tasks, 35 cells, image-only interface, stream separation and scoring eligibility. Historical authorization text is not an instruction to begin a run.

C5 - Synthetic two-frame renderers. `PreAttentiveVision/neuroscience_stimuli.py`: `CardinalMotionStream` and `TaskStream`. Defines cardinal motion, signed orientation, contrast, spatial frequency, chromatic increment and contour grouping.

C6 - Photograph spectral task. `PreAttentiveVision/natural_stimuli.py`: `NaturalSpectrum`. Defines source partitions, crop generation and frequency manipulation.

C7 - Ring orientation. `WorkingMemory/PlainBaseline/variants.py`: `VariantStream`. The suite uses its ring variant only; other diagnostic rungs are not quietly added to the training task list.

C8 - Spatial and sequence tasks. `WorkingMemory/SpatialTaskBattery/stimuli.py`: `SpatialBatteryStream`, `RecognitionImages`, `frame_count`. Defines signed cued orientation, motion duration, Krauzlis change, binding and recognition. The motion schedule helper is in `WorkingMemory/stimuli.py`.

D1 - New branch, design only. SecondPass/SpatialReadout/BRIEF.md, read 24 September 2026, together with the user's clarification that another agent is building the ConvGRU and it is not implemented yet. The reading treats it as a specification and does not infer launch or success from its authorization language.

R1 - Supplied research note. *Spatial KDA memory: architecture and causal tests*, 23 September 2026, supplied as 87252611-f998-4054-8a87-a46d9a26e873.pdf. Corresponding source and static architecture audit are in SecondPass/JointTraining/TechnicalReport/architecture_microstimulation.md and architecture_metrics_notes.md. Its measured scores retain their original dates and evaluation roles; this textbook does not update them.

Primary literature and documentation

P1. Kimi Team (2025). *Kimi Linear: An Expressive, Efficient Attention Architecture*, pinned v1. Background for the KDA name and fine-grained gated-delta lineage. The visual implementation's details are grounded in C1.

P2. Ballas, Yao, Pal and Courville (2016). *Delving Deeper into Convolutional Networks for Learning Video Representations*. Primary architectural precedent for convolutional GRU recurrence on video features. The planned branch's exact specification is D1.

P3. PyTorch documentation. *GRU*, accessed 24 September 2026. Source for the vector GRU equation and reset-placement convention; this citation is not a claim that the project uses the newest documentation's package version.

P4. Arcizet and Krauzlis (2018). *Covert spatial selection in primate basal ganglia*. Experimental provenance for the local two-patch adaptation. C8, rather than the biological paper, is authoritative for our rasterization, compressed timing and label generation.

P5. Morgan, Albanna and Herman (2025). *A recurrent vision transformer shows signatures of primate visual attention*, pinned v1. Functional comparison for memory-guided spatial attention and causal attention-bias tests. It does not establish the same mechanism in our KDA learner.

What this edition establishes

This edition explains the implemented computation and task definitions, derives consequences of their equations, and teaches the pending architectural proposal. It distinguishes illustrative arithmetic, source-defined design and measured evidence throughout. Whether the proposed ConvGRU improves any task remains a question for the separately implemented and evaluated experiment.